

[← Back to blog](#)

[~/blog](#)

# The Tale of Meaningful Vectors: Contrastive Learning for Text Embeddings, Told with Pen and Paper

May 17, 2026 • 9 min read • By Mohammed Vasim



0



contrastive-learning

text-embeddings

deep-learning

On this page



Imagine you are the librarian of a vast, magical library. Books are scattered everywhere. Your job is to arrange them so that books on similar topics sit near each other. A visitor whispers a few words, and you must instantly retrieve the perfect book. How would you teach an apprentice to do this?

You would show them examples: “These two books are about the same thing – put them together. That one is unrelated – push it far away.” This, in essence, is contrastive learning. Today, we will build a text embedding model from scratch using exactly this idea. No black boxes. We will do the math with pencil and paper, trace every number, and then connect our tiny creation to the mighty models like SimCSE and Qwen3 Embedding that power modern retrieval. By the end, you will have walked the path from zero to advanced, and it will feel as natural as sorting that library.

## 1. The Magic Library: Why Embeddings Matter

In our digital library, every book is a sentence. We want to represent each sentence as a point in space – a **vector** – such that sentences with similar meaning lie close together, and dissimilar ones lie far apart. This is the job of a text embedding model.

But how do we train such a model? We need a teaching signal: "this sentence means roughly the same as that one" (positive pair) or "these two are unrelated" (negative pair). Contrastive learning is the art of sculpting the vector space with these signals.

Let's start with a tiny universe. We'll use a vocabulary of just a few words and sentences of only two or three words. We'll build a simple encoder, define a loss, and tune the numbers by hand.

## 2. Our Tiny Universe: Sentences, Words, and Vectors

We'll pretend we have the following word embeddings (initialised randomly, each word is a 2-dimensional vector):

| Word   | Embedding    |
|--------|--------------|
| a      | [0.10, 0.20] |
| cat    | [0.30, 0.40] |
| sleeps | [0.50, 0.10] |
| dog    | [0.35, 0.45] |
| runs   | [0.45, 0.15] |
| the    | [0.05, 0.05] |
| loud   | [0.60, 0.60] |
| noise  | [0.70, 0.20] |

Our encoder will be simplicity itself: a sentence's vector is the **average** of its words' embeddings. For example:

*Sentence A:* "a cat sleeps"

Words: a, cat, sleeps

Sum =  $[0.10+0.30+0.50, 0.20+0.40+0.10] = [0.90, 0.70]$

Average =  $[0.30, 0.2333]$  (Let's round to  $[0.30, 0.23]$ )

*Sentence B:* "a dog runs"

Sum =  $[0.10+0.35+0.45, 0.20+0.45+0.15] = [0.90, 0.80]$

Average =  $[0.30, 0.2667]$  ( $\approx [0.30, 0.27]$ )

These two sentences are similar (both about pets doing actions). Their vectors are already close! Now a completely unrelated sentence:

*Sentence C:* "the loud noise"

Sum =  $[0.05+0.60+0.70, 0.05+0.60+0.20] = [1.35, 0.85]$

Average =  $[0.45, 0.2833]$  ( $\approx [0.45, 0.28]$ )

Now we need a loss function that says: **Pull A and B even closer; push A and C far apart.**

---

### 3. The Heart of Contrast: InfoNCE Loss

The most popular contrastive loss for modern text embeddings is **InfoNCE** (Info Noise-Contrastive Estimation). It treats the problem as a classification: among one positive and many negative candidates, which one is the correct match?

The formula looks intimidating but is a gentle giant:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(q, k_{+})/\tau)}{\sum_{i=1}^K \exp(\text{sim}(q, k_{-i})/\tau)}$$

Let's unpack it with our example.

- $q$ : the **query** (our sentence A).
- $k_{+}$ : the **positive key** (sentence B, the correct match).
- $k_i$ : a set that includes the positive and all negatives. Here we will have just one negative, sentence C. So  $K = 2$  (B and C).
- $\text{sim}$ : a similarity function; we'll use **cosine similarity**.

- $\tau$ : a temperature parameter (a small number like 0.1) that sharpens the distribution.

### 3.1 Step-by-step by hand

First, compute the embedding vectors (we already did):

$$q = [0.30, 0.23], \quad k_+ = [0.30, 0.27], \quad k_- = [0.45, 0.28]$$

Cosine similarity between two vectors **a**, **b** is:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

Calculate dot products and norms.

For **q** and **k<sub>+</sub>**:

$$q \cdot k_+ = 0.30 \times 0.30 + 0.23 \times 0.27 = 0.09 + 0.0621 = 0.1521$$

$$\|q\| = \sqrt{0.30^2 + 0.23^2} = \sqrt{0.09 + 0.0529} = \sqrt{0.1429} \approx 0.378$$

$$\|k_+\| = \sqrt{0.30^2 + 0.27^2} = \sqrt{0.09 + 0.0729} = \sqrt{0.1629} \approx 0.404$$

$$\text{sim}(q, k_+) = \frac{0.1521}{0.378 \times 0.404} \approx \frac{0.1521}{0.1527} \approx 0.996$$

(These vectors are extremely similar – almost parallel – by construction.)

For **q** and **k<sub>-</sub>**:

$$q \cdot k_- = 0.30 \times 0.45 + 0.23 \times 0.28 = 0.135 + 0.0644 = 0.1994$$

$$\|k_-\| = \sqrt{0.45^2 + 0.28^2} = \sqrt{0.2025 + 0.0784} = \sqrt{0.2809} \approx 0.530$$

$$\text{sim}(q, k_-) = \frac{0.1994}{0.378 \times 0.530} \approx \frac{0.1994}{0.2003} \approx 0.995$$

Wait, both similarities are nearly 1? That's because our initial random vectors happened to be highly correlated. That's fine – the loss will still push them based on the labels. However, to better see the contrast, let's use a more realistic temperature  $\tau = 0.2$ .

Compute the exponentials:

$$\exp(\text{sim}(q, k_+)/\tau) = \exp(0.996/0.2) = \exp(4.98) \approx 145.5$$

$$\exp(\text{sim}(q, k_-)/\tau) = \exp(0.995/0.2) = \exp(4.975) \approx 144.7$$

Now the denominator:  $145.5 + 144.7 = 290.2$ .

The probability that the model assigns to the positive match is:

$$P(k_+) = \frac{145.5}{290.2} \approx 0.5012$$

Loss:

$$\mathcal{L} = -\log(0.5012) \approx 0.690$$

This loss says: the model is only 50% confident about the correct pair. It must learn to assign higher probability to the positive pair.

### 3.2 What does the gradient do?

The gradient of the InfoNCE loss with respect to the query  $q$  (and similarly for the keys) boils down to:

$$\frac{\partial \mathcal{L}}{\partial q} = \frac{1}{\tau} \left( \sum_i P(k_i) \cdot \frac{k_i}{\|k_i\|} - \frac{k_+}{\|k_+\|} \right) \cdot \left( \frac{I}{\|q\|} - \frac{qq^T}{\|q\|^3} \right)$$

This is messy to compute by hand, but the intuition is simple: **the gradient pulls the query towards the positive key and pushes it away from all negatives, weighted by how much the model currently confuses them.**

To avoid the heavy algebra, we can use a simpler contrastive loss that yields identical intuition: the **triplet loss** or the **contrastive loss** (Chopra et al.). But since the blog aims at InfoNCE, we can demonstrate the parameter update with a tiny surrogate model where we directly differentiate the loss with respect to the word embeddings.

Instead of doing the full matrix calculus, we can make an approximation: imagine our encoder is just a single linear layer that directly produces the final vector from

the average of word embeddings (which it is). The gradient with respect to a word embedding is simply the gradient of the loss with respect to the sentence vector, divided equally among its words.

We already computed the loss  $L = 0.690$ . To lower it, we need to increase the similarity between  $q$  and  $k_+$  and decrease the similarity between  $q$  and  $k_-$ .

A manual "gradient" step: we can nudge the sentence vectors by a small amount in the direction of the positive minus the negative. Specifically, we can adjust the query  $q$  towards  $k_+$  and away from  $k_-$ . Let's set learning rate  $\eta = 0.1$ .

Update rule (simplified from the gradient of the InfoNCE with cosine similarity):

$$q \leftarrow q + \eta \cdot \frac{1}{\tau} \left( \frac{k_+}{\|k_+\|} - \sum_i P(k_i) \frac{k_i}{\|k_i\|} \right)$$

But for pen and paper, we can just do a vector addition that makes intuitive sense: move  $q$  a little towards  $k_+$  and away from  $k_-$ .

Positive pull:  $\delta_+ = k_+ - q = [0.30 - 0.30, 0.27 - 0.23] = [0.00, 0.04]$ . Negative push:  $\delta_- = q - k_- = [0.30 - 0.45, 0.23 - 0.28] = [-0.15, -0.05]$ .

A combined nudge:  $\Delta q = \eta(\delta_+ + \delta_-) = 0.1 \times [-0.15, -0.01] = [-0.015, -0.001]$ . New  $q = [0.30 - 0.015, 0.23 - 0.001] = [0.285, 0.229]$ .

Now recompute similarity with  $k_+$  and  $k_-$ ; you'll see the gap widen. Repeat many times, and the positive pair dominates.

Now distribute this sentence-level gradient to the word embeddings. Each word in the sentence gets  $1/(\text{sentence length})$  of the gradient. We update each word's embedding accordingly. This is exactly how backpropagation tunes the parameters.

After thousands of such tiny updates across millions of sentence pairs, the word embeddings (and in a real model, the Transformer weights) learn to map semantically similar sentences close together.

## 4. The Research Paper That Changed the Game: SimCSE

If you want to point to one paper that made contrastive learning for text embeddings simple and spectacular, it's **SimCSE** (Simple Contrastive Learning of Sentence Embeddings) by Gao et al., 2021.

The genius of SimCSE lies in its **unsupervised** approach: take a sentence, pass it through a pre-trained language model **twice** with different dropout masks. The two resulting embeddings are treated as a positive pair. Why does this work? Because the model learns to ignore the randomness of dropout and focus on the stable semantic content. All other sentences in the batch become negatives.

This is beautifully elegant: no need for hand-crafted augmentations like word deletion or swapping. Dropout, already present in the Transformer, provides just enough noise. The loss function is exactly the InfoNCE we calculated, with cosine similarity and temperature.

SimCSE achieved state-of-the-art on the Semantic Textual Similarity (STS) benchmark, beating many supervised models. Its simplicity ignited a wave of research.

Key takeaway: The contrastive framework works as long as you have a smart way to generate positive pairs from unlabeled text. SimCSE used dropout; later works (like ConSERT, LaBSE) explored other augmentations.

---

## 5. The Apex: Qwen3 Embedding and the Modern Recipe

Now let's zoom out from our toy 2-dimensional library to the real thing. **Qwen3 Embedding** (2025) is one of the latest and greatest text embedding models, and its architecture and training directly build on the contrastive ideas we've just explored.

Here's how Qwen3 Embedding stands on the shoulders of giants and adds its own magic:

### 5.1 Decoder-only backbone

Unlike SimCSE's BERT (an encoder-only model), Qwen3 uses a **decoder-only Transformer** – the same architecture as GPT. It takes a sentence, appends a special `[EOS]` token, and then extracts the hidden state of that token from the last layer as the sentence embedding. This leverages the powerful sequence understanding of large language models.

## 5.2 Three-stage training

Qwen3 Embedding is not trained in one go. It follows a meticulous three-stage recipe:

1. **Weakly-supervised pre-training on massive synthetic data.** The team used their Qwen3 LLM to generate billions of (query, document) pairs across hundreds of languages and tasks. This diverse, synthetic dataset is perfect for contrastive learning because you automatically get positive pairs (the query and its generated answer) and in-batch negatives.
2. **Supervised fine-tuning on high-quality labeled data.** After the broad foundation, the model is refined using carefully curated human-labeled datasets (like MS MARCO) mixed with a small amount of the best synthetic data. This sharpens the embeddings for accurate retrieval.
3. **Model fusion.** Multiple fine-tuned checkpoints are merged using spherical linear interpolation (SLERP). This acts like a model soup, combining the strengths of different snapshots into a more robust final model.

## 5.3 Instruction awareness

Qwen3 Embedding can be steered with natural language instructions. For example, when embedding a query, you prepend: "Represent this sentence for searching relevant passages:" followed by the query. The document side remains instruction-free. This allows a single model to perform well on diverse tasks (classification, clustering, retrieval) just by changing the prefix.

## 5.4 Matryoshka Representation Learning (MRL)

Ever wanted to shrink your embeddings from 4096 dimensions to 256 without retraining? Qwen3 supports MRL, meaning the first few dimensions already contain



most of the semantic information. It's like a nesting doll – you can take the first 512 dimensions and still get decent results, saving storage in vector databases.

---

## 6. Putting It All Together: From Pen to Production

Your tiny pen-and-paper model is not so different from Qwen3 Embedding. Both do the same dance: take a query and candidates, compute similarity, and use the InfoNCE loss to pull matches together and push mismatches apart. The differences are scale and sophistication:

- **Encoder:** Our averaging trick became a giant Transformer.
- **Data:** Our three sentences became billions of synthetic and real pairs.
- **Training:** Our one gradient step became a three-stage pipeline with model fusion.
- **Inference:** Our 2-dim vector became a 4096-dim powerhouse, but can be trimmed thanks to MRL.

Contrastive learning is the quiet hero behind every modern semantic search system. The next time you type a query into a chatbot or a document search bar, remember that invisible librarian, tirelessly sorting vectors so your answer appears like magic.

---

## 7. Your Turn: Build Your Own Tiny Contrastive Learner

Now that you've seen the theory and the giants, try this yourself:

1. Take 5 short sentences, give each word a random 2-dim embedding.
2. Form positive pairs (sentences with similar meaning) and negative pairs (random).
3. Implement the averaging encoder on paper.
4. Compute the InfoNCE loss for a query with one positive and two negatives.
5. Update the word vectors using a simplified gradient direction (move query towards positive, away from negatives).

6. After a few iterations, watch how the sentence vectors cluster.

You'll have built a contrastive text embedding model from scratch, and you'll never look at a vector database the same way again.

---

*Happy learning, and may your vectors always be meaningful!*

## Comments (0)

No comments yet. Be the first to comment!

### Leave a comment

Name

Email

Comment

Post Comment